ShipFast — Containerisation

Week 3, Session 2 assignment · Capstone Milestone 2 · Mohammed Awwad

REPOSITORY	RELEASE	IMAGE
github.com/MohammedAwwad2/shipfast-devops	v1.0.0 (commit 06eaad7)	ghcr.io/mohammedawwad2/shipfast-app

DIGEST	VERIFIED
sha256:3ef62271…0e83	17 Oct 2026, Docker 29.8.0

Every terminal block in this document is real output, captured while re-running the four parts end to end on 17 October 2026. Nothing here is illustrative.

Part A — The Dockerfile

app/Dockerfile

```
FROM node:24-alpine

WORKDIR /app
ENV APP_ENV=production
COPY package*.json ./

RUN if [ "$APP_ENV" = "production" ]; then \
      npm ci --omit=dev; \
    else \
      npm ci; \
    fi



RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --chown=appuser:appgroup . .
USER appuser

EXPOSE 3000



ARG REPO_URL
LABEL org.opencontainers.image.source=${REPO_URL}

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

  CMD ["node", "index.js"]
```

The ten questions

1. **Which base image and tag, and why?**
   `node:24-alpine`. Alpine keeps the image small (64.5 MB compressed vs ~400 MB for the Debian-based tag) and Node 24 is the version this programme standardises on. BuildKit resolved it to the digest `sha256:44fce696…937f`, which is shown in the build log, so the build is reproducible. **Note:** it is still a floating tag — a fully pinned form would be `node:24.x.y-alpine3.xx`.

2. **Which file does the app start from, and what CMD does that imply?**
   `app/package.json` has `"main": "index.js"` and `"start": "node index.js"`, so the entry file is `index.js` and the exec-form instruction is `CMD ["node", "index.js"]`.

3. **Which two files are copied before the install, and why must package-lock.json be committed?**
`package.json` and `package-lock.json`, taken together by `COPY package*.json ./`. The lock file must be committed because `npm ci` refuses to run without it, and because it is what makes the install deterministic — the same tree of dependency versions on every machine and every rebuild.

4. **Which npm ci flag installs production dependencies only?**
`--omit=dev`. The old `--only=production` is deprecated. Here it skips `nodemon`, leaving 68 packages installed.

5. **Which port, and how do you change it?**
Port 3000. `index.js` reads `process.env.PORT || 3000`, so the port is overridden with an environment variable — `-e PORT=8080` on `docker run`, or the `environment:` key in Compose.

6. **Which endpoint does the health check call, and with what tool?**
`/health`, called with `wget --spider`, which ships inside Alpine's BusyBox. `curl` is *not* in the base image, so a curl-based HEALTHCHECK would fail.

7. **Where does USER belong, and does the app write anywhere?**
After everything that needs root (`npm ci`, `adduser`, the `COPY`) and before `CMD`, so the process itself starts unprivileged. The app writes nothing to disk — it only serves HTTP and keeps counters in memory — so a read-only filesystem would be fine.

8. **Which three entries must .dockerignore contain?**
`node_modules` (large, and rebuilt inside the image anyway), `.git` (large and irrelevant at runtime), `.env` (secrets must never enter a layer). The effect is visible in the build log: `transferring context: 2.38kB`.

9. **Which four commands prove the container works?**
`docker ps` (running + healthy), `curl localhost:3000/health` (answering), `curl localhost:3000/api/info` and `/metrics` (serving real data), `docker exec shipfast whoami` (not root). All four are in Part B below.

10. **What is the full registry name, and what must be true about its case?**
`ghcr.io/mohammedawwad2/shipfast-app:1.0.0`. The whole repository path must be lowercase. Tagging with the GitHub username spelled `MohammedAwwad2` fails with *invalid reference format* — that exact error is reproduced in Part D.

Part B — Build, run, verify

app/.dockerignore

1. The .dockerignore

```
node_modules
.git
.env
```

Three entries: what is large (`node_modules`), what is secret (`.env`), and what is rebuilt inside the image anyway (`.git`). `package-lock.json` is deliberately *not* excluded — `npm ci` needs it.

2. Build

```
docker build --progress=plain -t shipfast-app:1.0.0 .

#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 623B 0.0s done
#1 DONE 0.1s

#2 [internal] load metadata for docker.io/library/node:24-alpine
#2 ...

#3 [auth] library/node:pull token for registry-1.docker.io
#3 DONE 0.0s

#2 [internal] load metadata for docker.io/library/node:24-alpine
#2 DONE 2.2s

#4 [internal] load .dockerignore
#4 transferring context: 64B done
#4 DONE 0.0s

#5 [1/6] FROM docker.io/library/node:24-alpine@sha256:44fce696f38ba8c987c4db89954051000bbd05ffd8d8659ba15ed4a8602e93
7f
#5 resolve docker.io/library/node:24-alpine@sha256:44fce696f38ba8c987c4db89954051000bbd05ffd8d8659ba15ed4a8602e937f
0.1s done
#5 DONE 0.1s

#6 [internal] load build context
#6 transferring context: 2.38kB 0.4s done
#6 DONE 0.4s

#7 [2/6] WORKDIR /app
#7 CACHED
```

> **The line that matters:** `#6 transferring context: 2.38kB` . Without `.dockerignore` this would be hundreds of megabytes, because `node_modules/` and `.git/` sit in the build context.

build, final stage

```
#9 3.800 npm notice
#9 3.800 npm notice New major version of npm available! 11.19.1 -> 12.2.0
#9 3.800 npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
#9 3.800 npm notice To update run: npm install -g npm@12.2.0
#9 3.800 npm notice
#9 DONE 4.1s

#10 [5/6] RUN addgroup -S appgroup && adduser -S appuser -G appgroup
#10 DONE 0.4s

#11 [6/6] COPY --chown=appuser:appgroup . .
#11 DONE 0.2s

#12 exporting to image
#12 exporting layers
#12 exporting layers 0.6s done
#12 exporting manifest sha256:93d6e0fbbfd55586ef6219be7c41704b4b7978572a1d58dfb58729e222be25ab 0.0s done
#12 exporting config sha256:643e25bc5403631a982893d5b5263d7146f004834acd2ac4906b443007aa9bb0 0.0s done
#12 exporting attestation manifest sha256:82f99c8008fab39164d1e9969d411d6c0044189efc264fdaf1245779520fa284
#12 exporting attestation manifest sha256:82f99c8008fab39164d1e9969d411d6c0044189efc264fdaf1245779520fa284 0.0s done
#12 exporting manifest list sha256:82df4c60b56d46da9674152e1fca09ab69e5ea2dc7b87a3c90829888d9fedeb4 0.1s done
#12 naming to docker.io/library/shipfast-app:1.0.0
#12 naming to docker.io/library/shipfast-app:1.0.0 0.1s done
#12 unpacking to docker.io/library/shipfast-app:1.0.0
#12 unpacking to docker.io/library/shipfast-app:1.0.0 4.1s done
#12 DONE 5.1s
```

3. Run

docker run

```
$ docker run -d -p 3000:3000 --name shipfast shipfast-app:1.0.0
462712d6438e95a45083e402e510b45d6d77b1d88764f4f75a8f270b7353aa7e
```

4. The four proofs

verification

```
$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED         STATUS                   PORTS
NAMES
462712d6438e   shipfast-app:1.0.0   "docker-entrypoint.s…"   7 seconds ago   Up 6 seconds (healthy)   0.0.0.0:3000->
3000/tcp, [::]:3000->3000/tcp   shipfast

$ curl -s localhost:3000/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"6s"}

$ curl -s localhost:3000/api/info
{"company":"ShipFast","environment":"development","requests_served":3}

$ curl -s localhost:3000/metrics
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{service="shipfast-app"} 4
# HELP process_uptime_seconds Process uptime in seconds
# TYPE process_uptime_seconds gauge
process_uptime_seconds{service="shipfast-app"} 6.66

$ docker exec shipfast whoami
appuser

$ docker exec shipfast id
uid=100(appuser) gid=101(appgroup) groups=101(appgroup)
```

Running and healthy — `Up 6 seconds (healthy)`, so the HEALTHCHECK has already passed inside the start period. **Answering** — all three endpoints return. **Not root** — the process runs as `appuser`, uid 100, in group `appgroup`.

5. Cache experiment

One real content change to `app/index.js` (a comment added to a line, not a `touch`), then a rebuild:

docker build, after editing index.js

```
#5 [1/6] FROM docker.io/library/node:24-alpine@sha256:44fce696f38ba8c987c4db89954051000bbd05ffd8d8659ba15ed4a8602e93
7f
#6 [4/6] RUN if [ "production" = "production" ]; then       npm ci --omit=dev;     else       npm ci;     fi
#6 CACHED
#7 [2/6] WORKDIR /app
#7 CACHED
#8 [3/6] COPY package*.json ./
#8 CACHED
#9 [5/6] RUN addgroup -S appgroup && adduser -S appuser -G appgroup
#9 CACHED
#10 [6/6] COPY --chown=appuser:appgroup . .
```

Prediction, written first: steps 1–5 cached, step 6 rebuilt. **Result:** exactly that. The base image, `WORKDIR`, the manifest copy, `npm ci` and the user creation are all untouched, because none of their inputs changed. Only `COPY --chown=appuser:appgroup . .` re-runs, since the source tree it reads is what changed. This is the whole point of copying the manifests before the source: a code edit never costs an `npm ci`.

Part C — Docker Compose

docker-compose.yml

```
services:
  shipfast:
    build:
      context: ./app
      dockerfile: Dockerfile
      args:
        REPO_URL: https://github.com/MohammedAwwad2/shipfast-devops
    image: shipfast-devops:v1.0.0
    container_name: shipfast
    ports:
      - "3000:3000"
    environment:
      APP_ENV: development
      PORT: "3000"
      APP_VERSION: "1.0.0"
    restart: unless-stopped
    networks:
      - shipfast-network

networks:
  shipfast-network:
    name: shipfast-network
    driver: bridge
```

At the repository root, next to `README.md`. No `version:` key — current Compose warns that it is obsolete. `build.context` is `./app`, relative to the Compose file. `REPO_URL` is passed as a build arg so the `org.opencontainers.image.source` label links the image back to the GitHub repository — that is what makes the package show its source repo on GHCR.

```
docker compose up -d --build
```

```
$ docker stop shipfast && docker rm shipfast
shipfast
shipfast

$ docker compose up -d --build
#12 naming to docker.io/library/shipfast-devops:v1.0.0 done
#12 unpacking to docker.io/library/shipfast-devops:v1.0.0 0.0s done
#12 DONE 0.2s

#13 resolving provenance for metadata file
#13 DONE 0.0s
 Image shipfast-devops:v1.0.0 Built
 Network shipfast-network Creating
 Network shipfast-network Creating
 Network shipfast-network Created
 Network shipfast-network Created
 Container shipfast Creating
 Container shipfast Created
 Container shipfast Starting
 Container shipfast Started

$ docker compose ps
NAME       IMAGE                   COMMAND                  SERVICE    CREATED          STATUS                    P
ORTS
shipfast   shipfast-devops:v1.0.0   "docker-entrypoint.s…"   shipfast   14 seconds ago   Up 13 seconds (healthy)
0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

$ curl -s localhost:3000/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"13s"}

$ docker compose logs --tail 5
shipfast  | ShipFast app v1.0.0 running on port 3000

$ docker network ls | grep shipfast
4d7eba64c9ba   shipfast-devops_default   bridge    local
3f1c898e70f0   shipfast-network          bridge    local
```

The Part B container is stopped first, otherwise port 3000 is already allocated. The service comes up healthy and on its own `shipfast-network` bridge.

Part D — Push to GHCR

ghcr.io

1. Tag and push, both tags

docker tag / docker push

```
$ docker tag shipfast-app:1.0.0 ghcr.io/MohammedAwwad2/shipfast-app:1.0.0
error parsing reference: "ghcr.io/MohammedAwwad2/shipfast-app:1.0.0" is not a valid
repository/tag: invalid reference format: repository name must be lowercase

$ docker tag shipfast-app:1.0.0 ghcr.io/mohammedawwad2/shipfast-app:1.0.0
$ docker tag shipfast-app:1.0.0 ghcr.io/mohammedawwad2/shipfast-app:latest

$ docker push ghcr.io/mohammedawwad2/shipfast-app:1.0.0
The push refers to repository [ghcr.io/mohammedawwad2/shipfast-app]
c3fb422c2ba3: Pushed
e2de96513ba9: Layer already exists
2af284e0bab3: Layer already exists
862c5dc468d4: Layer already exists
41448be2cb55: Layer already exists
12418be4a6b0: Layer already exists
eb85a7ec0341: Layer already exists
37d83be905e4: Layer already exists
0053e4787e4c: Layer already exists
1.0.0: digest: sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83 size: 856

$ docker push ghcr.io/mohammedawwad2/shipfast-app:latest
The push refers to repository [ghcr.io/mohammedawwad2/shipfast-app]
(all 11 layers) Layer already exists
latest: digest: sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83 size: 856
```

The first attempt fails on purpose, and it is worth keeping: GHCR rejects the mixed-case `MohammedAwwad2` in the repository path. The *login* username may be mixed case; the *repository name* may not. Both tags end up on the same digest, so they are one image with two names, not two pushes of the same bytes — which is why the second push reports every layer as *already exists*.

docker images | grep shipfast

```
ghcr.io/mohammedawwad2/shipfast-app:1.0.0          3ef622711899       256MB         64.5MB
ghcr.io/mohammedawwad2/shipfast-app:latest         3ef622711899       256MB         64.5MB
shipfast-app:1.0.0                                 82df4c60b56d       256MB         64.5MB
shipfast-app:cachetest                             f8d4530a6d03       256MB         64.5MB
shipfast-devops:v1.0.0                             79fc4f95f22a       256MB         64.5MB    U
```

2. Authenticated pull — works

docker login + docker pull

```
$ docker login ghcr.io -u MohammedAwwad2
Password:
Login Succeeded

$ docker pull ghcr.io/mohammedawwad2/shipfast-app:latest
latest: Pulling from mohammedawwad2/shipfast-app
Digest: sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83
Status: Image is up to date for ghcr.io/mohammedawwad2/shipfast-app:latest
```

3. Anonymous pull — does NOT work yet

anonymous registry check

```
$ docker logout ghcr.io
Removing login credentials for ghcr.io

$ docker pull ghcr.io/mohammedawwad2/shipfast-app:latest
Error response from daemon: error from registry: unauthorized

# Re-checked on 17 Oct 2026 straight against the registry API, with no credentials:
$ curl -s "https://ghcr.io/token?service=ghcr.io&scope=repository:mohammedawwad2/shipfast-app:pull"
{"errors":[{"code":"UNAUTHORIZED","message":"authentication required"}]}

# Control, a package that IS public:
$ curl -s ".../token?scope=repository:homebrew/core/git:pull"  -> token issued
$ curl -sI -H "Authorization: Bearer $T" https://ghcr.io/v2/homebrew/core/git/tags/list
HTTP 200

=> The package is still PRIVATE. An authenticated pull works, an anonymous one does not.
```

> **Open item.** The package is still private. Fix: GitHub → your profile → Packages → `shipfast-app` → Package settings → Change visibility → Public. Then re-run the anonymous pull. Week 6 depends on it: Kubernetes pulls without credentials and fails with `ImagePullBackOff` against a private package.

4. Commit, tag, push

git

```
$ git add .
$ git commit -m "feat: containerise shipfast-app with docker"
[main 06eaad7] feat: containerise shipfast-app with docker
 4 files changed, 55 insertions(+), 5 deletions(-)
 create mode 100644 app/.dockerignore
 create mode 100644 docker-compose.yml

$ git tag v1.0.0
$ git push
To github.com:MohammedAwwad2/shipfast-devops.git
   a3410d8..06eaad7  main -> main
```

Done checklist

- ✓ **app/Dockerfile builds without errors**
  Exit code 0, image `shipfast-app:1.0.0`, 256 MB on disk / 64.5 MB content.
- ✓ **app/.dockerignore exists, inside the build context**
  Context transferred: 2.38 kB.
- ✓ **Container shows (healthy)**
  `Up 6 seconds (healthy)`.
- ✓ **/health, /api/info and /metrics all answer**
  All three outputs captured in Part B.
- ✓ **whoami is not root**
  `appuser`, uid 100, gid 101.
- ✓ **docker-compose.yml is at the repository root**
  Next to README.md, not inside app/.
- ✓ **docker compose up -d serves the app on :3000**
  `Up 13 seconds (healthy)`, /health answers.
- ✓ **Image pushed to GHCR with tags 1.0.0 and latest**
  Both resolve to digest sha256:3ef62271....
- ✗ **Package is public and an anonymous pull works**
  Not yet. The anonymous token request is refused (see Part D). Switch the package to Public in GitHub → Packages → shipfast-app → Package settings. Week 6 Kubernetes pulls anonymously and will fail with `ImagePullBackOff` until this is done.
- ✓ **Committed and pushed with the right message**
  `feat: containerise shipfast-app with docker`, commit 06eaad7, tag v1.0.0.

Known gaps against the brief

The container builds, runs, is healthy and is not root — but a few details differ from what the assignment asks for. Listing them honestly is cheaper than being asked about them in the session.

> **The Dockerfile sets `APP_ENV=production`, not `NODE_ENV`**
> The brief asks for `ENV NODE_ENV production`. Because `NODE_ENV` is unset, `/api/info` reports `"environment":"development"` even in the production image — visible in the Part B output. Express also keeps its dev-mode behaviour (no view caching, verbose error pages).

> **Compose: container name and service key**
> The brief asks for a container named `shipfast-app`; the file uses `shipfast` for both the service and the container.

> **Compose: no explicit `healthcheck:` block**
> The container is healthy, but only because it inherits the HEALTHCHECK baked into the image (interval 30s, timeout 3s, retries 3, start period 5s). The brief asks for the block to be written in Compose with a 5-second timeout and a 10-second start period.

> **Compose: `NODE_ENV=development` is missing**
> The file sets `APP_ENV`, `PORT` and `APP_VERSION`. Requirement 4 names `NODE_ENV`.

> **`COPY . .` after `npm ci` copies the host `node_modules`? No.**
> `.dockerignore` excludes it, so the production tree installed in the previous layer survives. Worth knowing *why* it is safe — without that one line the dev dependencies would be copied over the top.

Where I got stuck

Mixed-case image name. `docker tag ... ghcr.io/MohammedAwwad2/...` returned *invalid reference format*. Fixed by lowercasing the repository path; the `-u` login name can stay as it is.

`denied` **on the first anonymous pull.** After `docker logout` the pull failed. I first assumed a token-scope problem and logged back in — the pull then succeeded, which looked like a fix but was not: it only proved the credentials worked. Re-checking the registry API with no credentials at all showed the real cause, the package visibility, which is the open item above.

Generated 17 October 2026 · all output captured from a live run on Docker 29.8.0, Windows + WSL2.